Homework 1
Network Dynamics and Learning

Pardis Poostpardis 343187
Yasaman Noshirvanbaboli 327167
Farnood Sotoudeh 341275
Sina Hamedani 315495

Introduction

Graph theory, a key mathematical discipline, is integral to a myriad of fields, offering essential insights and modeling capabilities. One notable application is in social network analysis, where it helps map and analyze complex relationships. In these models, vertices represent individuals or entities, and edges signify the connections or interactions between them. This methodology is crucial for understanding social dynamics, identifying influential nodes, and exploring network structures. Such applications highlight graph theory's versatility in dissecting and interpreting intricate relational patterns.

Requirements

Python libraries used in this project include `NetworkX`, `NumPy`, `SciPy`, `Matplotlib`, and `CVXPY`. These libraries provide the mathematical and computational tools necessary for network modeling, visualization, and optimization.

1 Exercise 1

Consider the network in Figure 1 with the following link capacities:

$$c_1 = c_2 = c_3 = c_4 = 3, \quad c_5 = 2, \quad c_6 = 1$$

1.1 Algorithms

`NetworkX` offers a wide range of network-related algorithms that enable complex graph analysis in a simple and efficient way. In this project, I use the following functions from the `nx.algorithms.flow` module:

- **minimum_cut:** Computes the value and the node partition of a minimum (o, d)-cut.
- **maximum_flow:** Determines the flow that routes as much capacity as possible from the source to the sink, i.e., finds the flow $f_{\max}$ with maximum value.

1.2 Main Theorem

In the given scenario, it can be demonstrated that the value of any flow passing through a network is always less than or equal to the capacity of any o-d cut. Additionally, there exists a flow with the maximum value and a cut with the minimum capacity.
The primary theorem establishes the relationship between the maximum flow and the minimum cut of the network. This is known as the **Max-Flow Min-Cut Theorem**, which states:

$$\boxed{\text{Maximum value of } (o,d)\text{-flow} = \text{Minimum capacity among all } (o,d)\text{-cuts.}}$$

1.3 Data Preparation

To prepare the data, first created the network graph using `NetworkX`. Each edge was labeled with a `capacity` attribute as shown in Figure 1. These capacities are used by the algorithm to determine the maximal flow between the source and the sink.

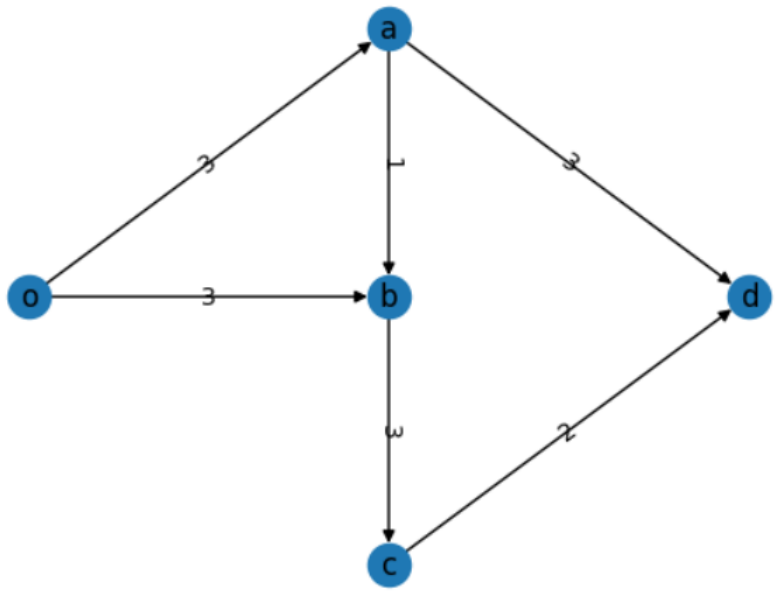


Figure 1: Network graph with labeled capacities.

Exercise 1

part a: Compute the capacity of all the cuts and find the minimum capacity to be removed for no feasible flow from o to d to exist.
We will determine the **maximum flow** from the source o to the sink d. This is one of the fundamental problems in network theory, typically solved using the *Maximum Flow* and *Minimum Cut* theorems, which are among the most important tools in optimization and resource allocation problems.
In this part of the exercise, we calculate all the possible cuts in the network. By evaluating their capacities, we identify the cut with the minimum total capacity. According to the **Max-Flow Min-Cut Theorem**, the value of this minimum cut is equal to the **maximum flow** that can be achieved through the network from o to d:

$$\boxed{f_{\max} = C_{\min}}$$

This relationship allows us to determine the highest possible amount of flow that can be transmitted from the source node to the sink node while satisfying all edge capacity constraints.
The approach involves calculating the capacities of all possible cuts in the network and identifying the one with the smallest total capacity, known as the **minimum cut**.
The minimum cut represents the set of edges whose total capacity is minimal among all cuts separating the source o and the sink d. Removing these edges would disconnect o from d, implying that no feasible flow can exist between them.
Using the built-in `NetworkX` functions, we can compute both the maximum flow value and the corresponding minimum cut partition efficiently. This computational approach confirms the theoretical relationship established by the Max-Flow Min-Cut Theorem.

Table 1: All possible cuts and their capacities.

No.	**Cut Partition** (S, T)	**Cut Capacity**
1	({o}, {a, b, c, d})	6
2	({a, o}, {b, c, d})	7
3	({o, b}, {a, c, d})	6
4	({o, c}, {a, b, d})	8
5	({a, o, b}, {c, d})	6
6	({a, o, c}, {b, d})	9
7	({o, b, c}, {a, d})	5
8	({a, o, b, c}, {d})	5

From Table 1, we observe that the minimum cut capacity is:

$$(S_{\min}, T_{\min}) = (\{a, o, b, c\}, \{d\})$$

$$(S_{\min}, T_{\min}) = (\{o, b, c\}, \{a, d\})$$

Both cuts have the same **capacity** of 5, and both minimum cuts involve the edge $c \to d$ (e5). When the `nx.algorithms.flow.minimum_cut()` method is applied, it selects the **first cut**.

$$\text{Chosen Cut: } \{o, b, c, a\}, \{d\}, \quad C_u = 5$$

According to the Max-Flow Min-Cut Theorem:

$$f_{\max} = C_{\min} = 5$$

Hence, the **maximum flow value** in the network is 5, which equals the minimum cut capacity.

Table 2: Edges in the minimum cut and their capacities

Edge	**Capacity**
$a \to d$	3
$c \to d$	2

The edges listed in Table 2 form the minimum cut. Removing these edges disconnects the source o from the sink d, which means no feasible flow can pass through the network.

Exercise 1

part b: Given $x > 0$ extra units of capacity ($x \in \mathbb{Z}$), determine how to distribute them in order to maximize the throughput from o to d, and plot the maximum throughput as a function of $x \geq 0$.
The problem now is to determine how to distribute $x > 0$ additional units of capacity across the network in order to maximize the total throughput from the source o to the sink d.
To address this, the `NetworkX` library was used to simulate the incremental addition of capacity to the network and compute how the maximum flow changes for each unit of extra capacity. The main idea is to identify the edges that currently form the **bottleneck** (those in the minimum cut) and strengthen them to achieve the greatest improvement in total flow. In this task, the **linking edges**—the edges that belong to the minimum cut and connect the two partitions of the graph—are the most effective locations to allocate the additional capacity. To identify these edges, the `NetworkX` minimum cut function was used, which returns the node partitions and the corresponding set of crossing edges.
After determining the linking edges, one unit of capacity was added to one of these edges during each iteration. This process was repeated until all extra capacity units were distributed. At every step, the algorithm increased the capacity of the most constraining linking edge to achieve the highest possible improvement in the network's total throughput.
For demonstration purposes, a total of $x = 10$ additional capacity units was used. After several iterations, the results indicated that:

$$(o, a) \text{ was incremented 5 times, and } (a, d) \text{ was incremented 6 times.}$$

This adaptive allocation produced the greatest increase in overall throughput from the source o to the sink d, confirming that reinforcing the linking edges of the minimum cut is the optimal strategy for maximizing network flow.
Figure 2 shows the maximum throughput from o to d with extra capacity added to the network, demonstrating how the flow increases as additional capacity is introduced.

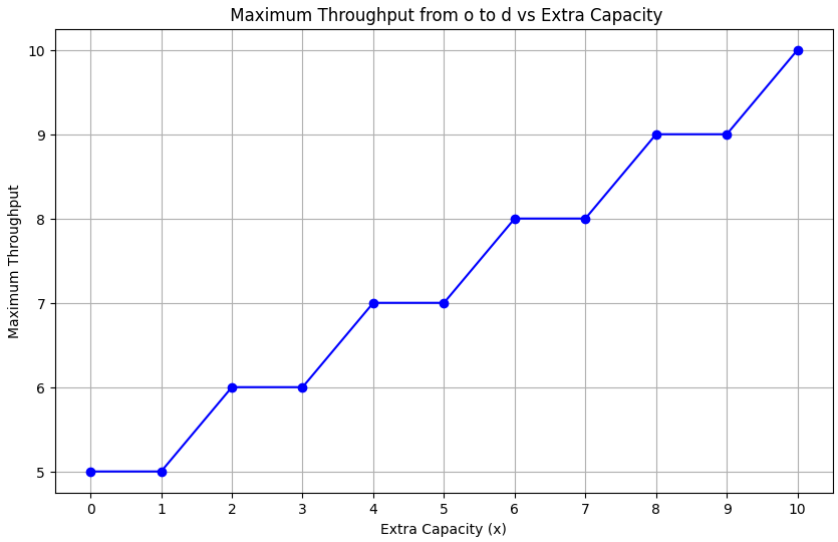


Figure 2: maximum throughtput

To increase the flow from the source o to the sink d, we tested adding extra capacity to different edges in the network. First, we increased the capacity of the edge $c \to d$ (e5), which raised the flow to 6 when $x = 1$. Then, when $x = 2$, we increased the capacity of the edge $a \to d$ (e2), but the flow stayed at 6 because $c \to d$ was still the bottleneck. Finally, when $x = 3$, we added capacity to the edge $o \to a$ (e1), which increased the flow to 7.

When we increase the capacity of $c \to d$ (e5) by 1, we noticed that **just increasing** $c \to d$ **every time is not the best strategy**. This is because we should also be increasing the capacities of **previous edges** that contribute to the overall flow, like $o \to b$ and $b \to c$. If we focus only on $c \to d$, we miss the opportunity to improve other important paths. This results in suboptimal network performance.

Exercise 1

part c: You are given the possibility of adding to the network a directed link e_8 with capacity $c_8 = 1$ and $x > 0$ extra units of capacity ($x \in \mathbb{Z}$). Determine where to add this link and how to distribute the additional capacity to maximize the throughput from o to d. Plot the maximum throughput as a function of $x \geq 0$.

In this section, the network model was extended by introducing a new directed link e_8 with an initial capacity of $c_8 = 1$, aimed at improving the total throughput from o to d. The objective was to determine both the optimal position for this link and the most effective distribution of the additional capacity $x > 0$ to maximize the overall network flow. To achieve this, several potential placements for e_8 were tested and compared based on their resulting maximum flow values. The analysis revealed that the most advantageous configuration is obtained when the new link directly connects the source to the sink, (o, d). After identifying this optimal placement, incremental units of additional capacity were introduced to study their influence on the total throughput. The results confirmed that adding a direct link from o to d significantly enhances network performance, as it provides a shortcut that bypasses intermediate bottlenecks and substantially increases the achievable flow.

Figure 3 shows that the maximum throughput increases almost linearly with the number of extra capacity units x. This linear growth indicates that adding both the new link (o, d) and additional capacity effectively enhances the network's performance, allowing a proportional rise in flow until the network reaches its structural limits.

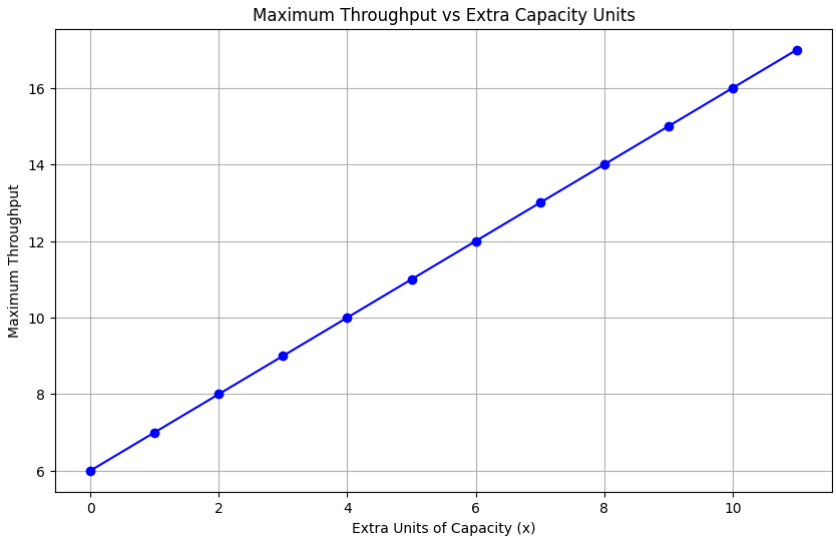


Figure 3: Maximum throughput from o to d as a function of extra capacity x after adding the new link e_8.

Exercise 2: Katz and PageRank Centrality on Graph G

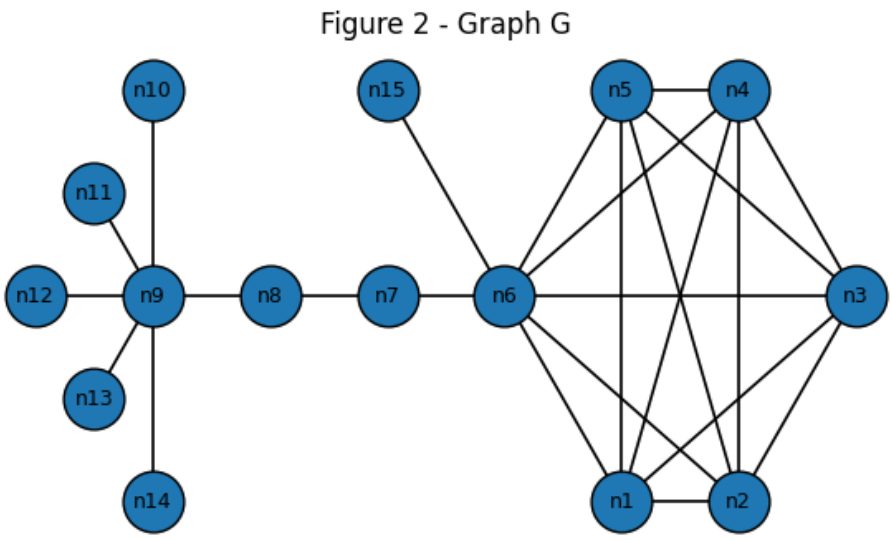


Figure 4: Graph $G = (V, E)$ used in Exercise 2.

Problem statement

The goal of this exercise is to analyze the structure of the directed graph $G = (V, E)$ in Figure 4 using two well–known centrality measures: **Katz centrality** and **PageRank**. Both assign a numeric "importance" to each node, but they are based on different underlying models.

- **Katz centrality** regards a node as important if it can be reached by many walks (of all lengths) from the rest of the network. Walks are exponentially damped with their length, so short paths contribute more than long ones. This measure is purely *algebraic*: it is defined directly in terms of the adjacency matrix.
- **PageRank** models a *random surfer* that repeatedly follows outgoing links with probability β and teleports to a random node with probability $1 - \beta$. Centrality is given by the steady–state

probability of this Markov chain. Here the transition matrix is column–stochastic and outgoing edges are weighted by the inverse out–degree.

Besides computing the two centralities, we are particularly interested in comparing the roles of nodes n_6 and n_9:

- n_6 is the articulation point connecting the dense clique $\{n_1, \ldots, n_5\}$ to the rest of the graph.
- n_9 is the center of a star with leaves $n_{10}, \ldots, n_{14}$ and is also connected to the path leading to the dense core.

We will show how these two nodes behave very differently under Katz and PageRank, and how their relative importance changes when the PageRank parameter β varies.

Part (a): Katz centrality, $\beta = 0.15$, uniform intrinsic centrality μ

Let A be the adjacency matrix of the directed graph G. Following the convention of the course, we take

$$A_{ij} = \begin{cases} 1, & \text{if there is an edge } j \to i, \\ 0, & \text{otherwise,} \end{cases}$$

so that column j encodes all edges leaving node j. Katz centrality with parameter $\beta > 0$ and intrinsic centrality vector $\mu \in \mathbb{R}^{|V|}$ is defined as the solution of

$$x = \beta A x + \mu \quad \Longleftrightarrow \quad (I - \beta A)x = \mu.$$

Assuming $\beta < 1/\lambda_{\max}(A)$, where $\lambda_{\max}(A)$ is the spectral radius of A, the matrix $I - \beta A$ is invertible and the unique solution is

$$x = (I - \beta A)^{-1}\mu.$$

In this exercise we use:

$$\beta = 0.15, \qquad \mu = \mathbf{1},$$

i.e., a uniform intrinsic centrality (all nodes receive the same base score) and a moderate damping factor. The value $\beta = 0.15$ is safely below the stability threshold $1/\lambda_{\max}(A)$, so the Katz vector is well defined.

A Python script constructs the graph, builds A according to a fixed node ordering $[n_1, \ldots, n_{15}]$, and solves the linear system above using `numpy`. The resulting scores, sorted in decreasing order, are reported in Table 3.

Table 3: Katz centrality ($\beta = 0.15$, uniform μ).

Node	**Katz score**
n_6	4.766901
n_1	4.287588
n_2	4.287588
n_3	4.287588
n_4	4.287588
n_5	4.287588
n_9	2.247507
n_7	1.959697
n_{15}	1.715035
n_8	1.631081
n_{10}	1.337126
n_{11}	1.337126
n_{12}	1.337126
n_{13}	1.337126
n_{14}	1.337126

Observation and structural interpretation. From Table 3 we can distinguish several groups of nodes:

- n_6 attains the *highest* Katz score. This is expected: any path that connects the dense clique $\{n_1, \ldots, n_5\}$ to the rest of the graph must go through n_6, and there are many short cycles involving n_6 and the clique. Hence n_6 receives contributions from a large number of short, highly weighted walks.
- The nodes $n_1, \ldots, n_5$ in the K_5 subgraph are symmetric and therefore obtain the same Katz value. Their score is only slightly lower than that of n_6, since they form a very dense region of the network.
- Node n_9 has a noticeably smaller score (about half of n_6). Although it is the center of a star with many leaves, most of the incoming walks that reach n_9 originate from those leaves, which are themselves poorly connected. In Katz centrality, paths starting from low–degree peripheral nodes are less influential than paths coming from the well–connected core.
- Nodes $n_{10}, \ldots, n_{14}$ are symmetric leaves of the star centered at n_9, hence they all share the same (low) score. Similarly, n_7, n_8, and n_{15} lie on the path or small branch connecting n_6 and n_9 and therefore have intermediate Katz values.

In particular,

$$x_{\text{Katz}}(n_6) = 4.766901, \qquad x_{\text{Katz}}(n_9) = 2.247507,$$

so that

$$x_{\text{Katz}}(n_6) - x_{\text{Katz}}(n_9) = 2.519394.$$

Katz centrality therefore clearly favors n_6 over n_9: from a path–based viewpoint, the articulation node n_6 is much more central than the local hub n_9.

Part (b): Distributed PageRank with $\beta = 0.15$, uniform μ

We now consider PageRank, which is based on a random–walk interpretation. Let A be the adjacency matrix as before and define the column–stochastic matrix P by normalizing each column:

$$P_{ij} = \frac{A_{ij}}{\text{outdeg}(j)}, \qquad \sum_i P_{ij} = 1 \quad \forall j.$$

The PageRank vector x with damping factor $\beta \in (0, 1)$ and teleportation vector μ satisfies

$$x = \beta P x + (1 - \beta)\mu,$$

that is,

$$(I - \beta P)x = (1 - \beta)\mu.$$

We use the standard choice of a uniform teleportation vector,

$$\mu = \frac{1}{N}\mathbf{1}, \qquad N = |V| = 15,$$

so that each node is equally likely when a teleportation event occurs.

Distributed message–passing algorithm. Instead of solving the linear system centrally, we implement PageRank via a distributed, iterative scheme. Each node maintains a local estimate $x_i(k)$ of its PageRank at iteration k and updates it using only information from its neighbors:

$$x_i(k{+}1) = \beta \sum_{j \in \text{In}(i)} \frac{x_j(k)}{\text{outdeg}(j)} + (1 - \beta)\mu_i.$$

In matrix form this is exactly the power–iteration method:

$$x(k{+}1) = \beta P x(k) + (1 - \beta)\mu.$$

The implementation follows the usual PageRank convention for *dangling nodes* (nodes with out–degree zero): their probability mass is redistributed according to μ, so that the transition matrix remains stochastic.

The algorithm proceeds as follows:

1. Initialize $x_i(0) = 1/N$ for all nodes.
2. At each step k:
   (a) Each node j sends the value $x_j(k)/\text{outdeg}(j)$ to all its out–neighbors.
   (b) Each node i sums the contributions received from its in–neighbors to form the quantity $\sum_{j \in \text{In}(i)} x_j(k)/\text{outdeg}(j)$.
   (c) Each node updates its value according to the local rule above.
3. Iterate until the ℓ_1–norm of the difference between successive vectors is below a tolerance (10^{-12} in our code).

To validate the distributed implementation, we also compute PageRank using a centralized power iteration based directly on the matrix P. The maximum absolute difference between the two solutions is on the order of 10^{-15}, which is essentially numerical noise, confirming that the distributed algorithm converges to the correct stationary distribution.

The resulting PageRank scores for $\beta = 0.15$ are summarized in Table 4.

Table 4: PageRank ($\beta = 0.15$, uniform μ) via distributed message passing.

Node	**PageRank**
n_9	0.10595747
n_6	0.08011496
n_1	0.06663448
n_2	0.06663448
n_3	0.06663448
n_4	0.06663448
n_5	0.06663448
n_8	0.06405467
n_7	0.06318752
n_{10}	0.05931560
n_{11}	0.05931560
n_{12}	0.05931560
n_{13}	0.05931560
n_{14}	0.05931560
n_{15}	0.05833842

Observation and structural interpretation. PageRank produces a ranking very different from Katz. The most striking difference is that

$$\text{PR}(n_9) = 0.1059 \quad \text{and} \quad \text{PR}(n_6) = 0.0801,$$

so PageRank now favors n_9 over n_6:

$$\text{PR}(n_6) - \text{PR}(n_9) = -0.0258 < 0.$$

The reason lies in the normalization by out–degree: n_9 receives almost all the probability mass of its five leaves $n_{10}, \ldots, n_{14}$, each of which has only one outgoing edge. In contrast, n_6 has a high out–degree (it is connected to $n_1, \ldots, n_5$, n_7, and n_{15}), and therefore spreads its outgoing probability among many neighbors. This reduces the fraction of steady–state mass that n_6 can retain.

The nodes in the clique $\{n_1, \ldots, n_5\}$ again share the same value by symmetry, and all leaves $n_{10}, \ldots, n_{14}$ have the same PageRank. Notice that their PageRank is not extremely small; thanks to teleportation they still receive some mass even though they have degree 1.

Part (c): Direct comparison of n_6 and n_9

Table 5 summarizes the values of n_6 and n_9 for both centrality measures at $\beta = 0.15$.

Table 5: Direct comparison of n_6 and n_9.

Metric	n_6	n_9	**Difference** ($n_6 - n_9$)
Katz ($\beta = 0.15$)	4.766901	2.247507	2.519394
PageRank ($\beta = 0.15$)	0.08011496	0.10595747	-0.02584251

The two measures emphasize different structural aspects:

- **Katz centrality.**
  - Counts *all* walks, but exponentially weights short ones more than long ones.
  - Rewards nodes that lie on many short paths between other well–connected nodes.
  - Strongly favors n_6, which sits between the dense clique and the rest of the graph and is involved in many short cycles with highly central neighbors.
- **PageRank.**
  - Considers the *steady–state probability* of a random walk with teleportation.
  - The contribution that a node sends to its neighbors is divided by its out–degree; a node with many outgoing edges shares its importance among many neighbors.
  - This penalizes n_6, which has high out–degree, and advantages n_9, which collects the full probability from its leaves, each with out–degree 1.

Thus, from a global connectivity perspective (Katz), n_6 is the most influential node in the graph, while from a probability–mass perspective (PageRank at $\beta = 0.15$), the local star hub n_9 becomes more important.

Part (d): PageRank for $\beta \in \{0, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1\}$ and monotonicity of $\Delta(\beta)$

We now study how the damping factor β influences the relative importance of n_6 and n_9. For each

$$\beta \in \{0, \tfrac{1}{4}, \tfrac{1}{2}, \tfrac{3}{4}, 1\},$$

we compute the PageRank vector $x(\beta)$ solving

$$x(\beta) = \beta P x(\beta) + (1 - \beta)\mu, \qquad \mu = \tfrac{1}{N}\mathbf{1},$$

and then evaluate the difference

$$\Delta(\beta) = \mathrm{PR}_\beta(n_6) - \mathrm{PR}_\beta(n_9).$$

The numerical results are displayed in Table 6.

Table 6: PageRank of n_6 and n_9 for different β values (uniform μ).

β	**PR**(n_6)	**PR**(n_9)	$\Delta(\beta)$ = **PR**(n_6) - **PR**(n_9)
0.00	0.06666667	0.06666667	0.00000000
0.25	0.08756064	0.12697773	-0.03947109
0.50	0.10140399	0.16669255	-0.06528856
0.75	0.11060449	0.19111594	-0.08011451
1.00	0.14583333	0.12500000	0.02083333

Key observations on the extremes.

- $\beta = 0$**.** In this case the random surfer always teleports and never follows edges. The equation reduces to $x(0) = \mu$, so every node has the same score $1/N$. Hence $\Delta(0) = 0$.
- $\beta \to 1$**.** As β approaches 1, teleportation becomes rare and the behavior is dominated by the random walk on the directed graph. In our case the graph is built by symmetrizing an undirected graph, so the stationary distribution of the random walk is proportional to the node degrees of the underlying undirected graph. In particular, $\deg(n_6) = 7$ (connections to $n_1, \ldots, n_5$, n_7, n_{15}) and $\deg(n_9) = 6$ (connections to n_8, $n_{10}, \ldots, n_{14}$), so n_6 is more likely than n_9 in the limit. This is reflected in the values for $\beta = 1$ in Table 6, where $\mathrm{PR}_1(n_6) > \mathrm{PR}_1(n_9)$ and thus $\Delta(1) > 0$.

Intermediate values and monotonicity. For intermediate β the difference $\Delta(\beta)$ is negative: the local star structure around n_9 dominates and attracts more probability mass than n_6. As β increases from 0 to 0.75, the magnitude of $\Delta(\beta)$ becomes more negative, indicating that n_9 becomes increasingly more important relative to n_6. However, when β reaches 1 the sign of $\Delta(\beta)$ flips and n_6 eventually overtakes n_9 because of its larger degree.

Therefore the sequence

$$\Delta(0),\ \Delta(1/4),\ \Delta(1/2),\ \Delta(3/4),\ \Delta(1)$$

is not monotone: it starts at 0, decreases to negative values, and finally becomes positive again. This sign change shows that the relative importance of n_6 and n_9 depends strongly on how much we emphasize teleportation versus pure random walk.

Summary and discussion. The analysis of this graph illustrates how Katz centrality and PageRank capture different notions of importance:

- Katz centrality favors nodes that are embedded in dense regions and that lie on many short paths between other nodes. In our graph this highlights the articulation node n_6 and the clique $\{n_1, \ldots, n_5\}$.
- PageRank distributes importance according to the equilibrium of a random walk with teleportation. Nodes with many low–degree in–neighbors receive a large fraction of the stationary mass, even if they are not globally well connected. This is the case for n_9, which absorbs almost all the probability of its five leaves.

For small and moderate values of β (strong teleportation), PageRank is dominated by local structures: n_9 is more central than n_6. For β close to 1, the random walk explores the global topology and the centrality becomes closer to degree centrality, so n_6 eventually becomes more important thanks to its higher degree and its role as bridge between the dense core and the periphery.

Overall, this exercise shows that:

- Centrality is inherently multi–scale: different measures and parameters emphasize local or global features of the same graph.
- Comparing Katz and PageRank on the same network helps to distinguish nodes that are *globally influential* (like n_6) from nodes that act as *local hubs* (like n_9).
- The parameter β in PageRank plays a crucial role: varying it can completely change the relative ordering of nodes, as illustrated by the non–monotonic behavior of $\Delta(\beta)$.

These observations highlight the importance of carefully choosing both the centrality measure and its parameters depending on the application and on the type of influence one wants to model.

Exercise 3

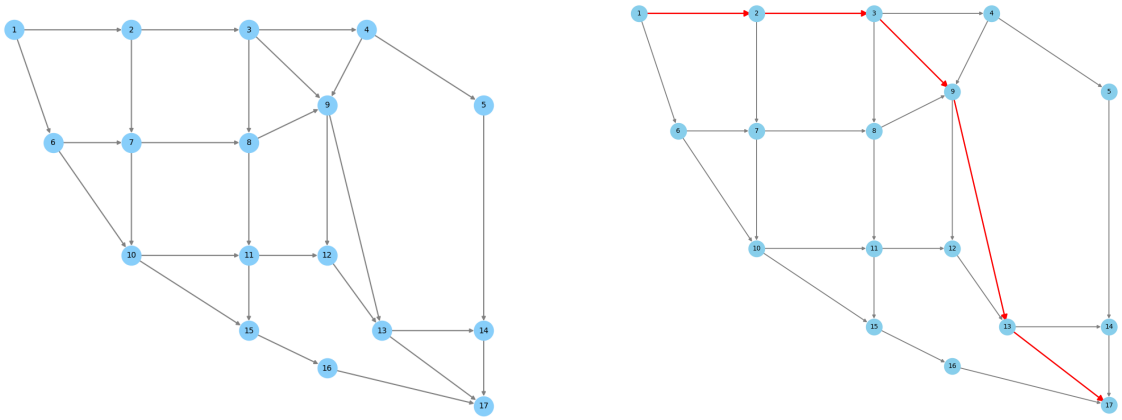
In this exercise, we analyze the provided Los Angeles highway network. We are given the node-link incidence matrix B, link capacities c_e, and free-flow travel times l_e. All analysis is performed using Python, with the ‘networkx‘ library for graph operations and ‘cvxpy‘ for optimization problems.

Part (a): Shortest path (empty network)

First, we find the fastest path from the source (node 1, Santa Monica) to the sink (node 17, Santa Ana). In an empty network, this is equivalent to finding the shortest path using the free-flow travel times l_e as the link weights.

We used the ‘networkx‘ library’s shortest path algorithm to solve this.

- **Shortest Path by Nodes:** The path follows the node sequence: $[\mathbf{1} \to \mathbf{2} \to \mathbf{3} \to \mathbf{9} \to \mathbf{13} \to \mathbf{17}]$
- **Shortest Path Length:** The total travel time (the sum of l_e) along this path is **0.559833**.



(a) The full Los Angeles highway network.

(b) The shortest path (1-17) highlighted in red.

Figure 5: Network analysis for Part (a): The full graph and the resulting shortest path.

Part (b): Maximum flow $1 \to 17$

Next, we determine the maximum flow the network can support from node 1 to node 17. This problem uses the link capacities c_e and is solved using the ‘networkx‘ implementation of the max-flow algorithm.

- **Maximum Flow Value:** The network can support a maximum flow of **22448.0** units.

Part (c): Feasibility Check and Demand ν

For this part, we were given a flow vector f in ‘flow.mat‘. We first had to compute the net flow vector $\nu = Bf$ to check the flow balance at each node.

A flow is feasible if it satisfies three conditions:

1. Flow f_e on each link is non-negative and does not exceed capacity c_e.
2. Flow is conserved at all internal nodes (i.e., $\nu_i = 0$ for $i \neq 1, 17$).
3. The net flow out of the source equals the net flow into the sink (i.e., $\nu_1 = -\nu_{17}$).

- **Feasibility Status:** The given flow f is **NOT FEASIBLE**.
- **Reason:** Our analysis showed it fails all three conditions. As shown in the vector below, flow is not conserved at internal nodes (e.g., $\nu_2 = 8570 \neq 0$). Furthermore, the source and sink flows do not balance, as $\nu_1 \neq -\nu_{17}$ ($16806 \neq -(-23544)$).

- **Net Flow Vector (ν):** The computed net flow $\nu = [\nu_1, ..., \nu_{17}]^T$ is:

$$\nu = \begin{bmatrix} 16806 \\ 8570 \\ 19448 \\ 4957 \\ -746 \\ 4768 \\ 413 \\ -2 \\ -5671 \\ 1169 \\ -5 \\ -7131 \\ -380 \\ -7412 \\ -7810 \\ -3430 \\ -23544 \end{bmatrix}$$

As per the assignment's instructions, this value $\nu_1 = 16806.0$ is used as the fixed demand D for all subsequent optimization problems (d, e, f, g).

Part (d): Social Optimum f^*

Here, we find the social optimum flow f^*. This is the flow that minimizes the total travel time experienced by all users in the system, $\sum_e f_e \tau_e(f_e)$, where τ_e is the non-linear delay function.

This is a convex optimization problem. We minimize the objective function provided in the assignment, which is equivalent to minimizing the total travel time:

$$\min \sum_{e \in \mathcal{E}} \left(\frac{l_e c_e}{1 - f_e/c_e} - l_e c_e \right)$$

Flow Constraint: We enforce a balanced flow constraint where the demand $D = 16806.0$ (from part c) leaves node 1 and enters node 17:

$$d_i = \begin{cases} 16806.0 & \text{if } i = 1 \text{ (Source)} \\ -16806.0 & \text{if } i = 17 \text{ (Sink)} \\ 0 & \text{otherwise} \end{cases}$$

Results:

- **Solver Status:** `optimal`
- **Note on Status:** Our initial attempt with the `SCS` solver returned an `optimal_inaccurate` status. While this is a common warning for this problem (due to high demand pushing links near capacity), it provided a slightly imprecise flow vector. To ensure our toll calculations in later parts were accurate, we switched to the `ECOS` solver, which is more robust for this class of problem and returned a definitive `optimal` status.
- **Total Travel Time:** The total travel time for all users at the social optimum is **26142.67**.
- **Optimal Flow f^*:** The resulting flow f^* is distributed across the entire network. For example, the flows on the first few links are [6569.22, 5809.94, 3047.02, ...].

Part (e): Wardrop Equilibrium Flow

The objective of Part (e) is to determine the Wardrop equilibrium flow $f^{(0)}$ for the network. At Wardrop equilibrium, no driver can reduce their travel time by changing routes, resulting in a balanced travel time across alternative paths.

We find the equilibrium by minimizing the Beckmann potential, which serves as the congestion cost function:

$$\sum_{e\in E}\int_0^{f_e}\tau_e(s)\,ds = \sum_{e\in E}\left(-l_e c_e \ln(c_e - f_e) + l_e c_e \ln(c_e)\right)$$

Results of the Wardrop equilibrium calculation (using the `ECOS` solver) are as follows:

- **Wardrop Optimum Flow** $f^{(0)}$: $[6559.13, 6311.39, 2199.11, 2199.11\ldots, 5293.73]$
- **Wardrop Cost:** 26495.32

We then compute the **Price of Anarchy (PoA)**, defined as the ratio of the Wardrop equilibrium cost to the social optimum cost from Part (d):

$$\text{PoA} = \frac{\text{Wardrop Cost}}{\text{Optimal Cost}} = \frac{26495.32}{26142.67} = 1.0135$$

The PoA is greater than 1, which correctly indicates that the network is inefficient when users route selfishly. The 1.35% inefficiency is what we aim to correct in the next part using tolls.

Part (f): Introducing Tolls to Achieve Social Optimum

In Part (f), we introduce tolls to align individual drivers' route choices with the system optimum. We determine tolls for each link e such that the new Wardrop equilibrium flow corresponds exactly to the system optimal flow f^* from Part (d).
The toll on each link e is defined by the marginal cost formula:

$$\omega_e = f_e^* \tau_e'(f_e^*)$$

where f_e^* is the social optimum flow on the link and $\tau_e'(f_e^*)$ is the derivative of the delay function.
We then solve for the new Wardrop equilibrium by minimizing the new perceived cost, which is the integral of (travel time + toll):

$$\sum_{e\in E}\int_0^{f_e}\left(\tau_e(s) + \omega_e\right)ds$$

This is equivalent to minimizing the original Beckmann potential (from Part e) plus the total toll revenue $\sum_e \omega_e f_e$.
Results obtained after solving for the Wardrop equilibrium under the computed tolls are as follows:

- **Wardrop equilibrium with tolls** ($f^{(\omega^*)}$): The resulting flow vector is identical to the social optimum flow f^* from Part (d): $[6569.22, 5809.94, 3047.02, 3047.02\ldots, 5040.94]$
- **Wardrop cost with tolls:** 26142.67
- **PoA with tolls:** 1.0000

The new Wardrop equilibrium flow and its total cost are now identical to the social optimum. The Price of Anarchy is exactly 1.0. This confirms that the marginal-cost tolls successfully internalize the congestion externality, forcing selfish drivers to collectively achieve the socially optimal outcome.

Part (g): Alternative Cost Function — Additional Travel Time

In this part, we redefine the system cost as the total additional travel time compared to the travel time under free-flow conditions. The new cost function for a link e is:

$$\tilde{\psi}_e(f_e) = f_e(\tau_e(f_e) - l_e)$$

The total additional travel time across the network becomes:

$$\text{New Cost Function} = \sum_{e\in E} f_e\left(\frac{l_e}{1 - f_e/c_e} - l_e\right) = \sum_{e\in E}\left(\frac{l_e c_e}{1 - f_e/c_e} - l_e c_e - l_e f_e\right)$$

Optimization Problem Setup: We first find the new system optimal flow, f^*_{new}, by minimizing this new objective function, subject to the same demand and flow constraints.

Results: The system optimal flow f^*_{new} and corresponding optimal cost were obtained as follows:

- **System Optimal Flow** (f^*_{new}): $[6584.51, 5577.70, 3367.56, 3367.55, \ldots, 4905.74]$
- **Optimal Cost (Additional Time):** 15350.35

(Note: The total travel time of this flow, $\sum f_e \tau_e(f_e)$, is 26186.39)

Toll Calculation: To align the Wardrop equilibrium with this new optimum, the tolls must be based on the marginal cost of this new objective function. The correct toll is:

$$\omega_e = f^*_e \cdot \tau'_e(f^*_e) - l_e$$

We calculate this new toll vector ω^* using the f^*_{new} flow.

Wardrop Equilibrium with Tolls: Finally, we verify this works by finding the Wardrop equilibrium under these new tolls. We again minimize the Beckmann potential plus the new toll revenue: $\sum \int \tau_e(s)ds + \sum \omega_e f_e$.

- **Wardrop Equilibrium Flow with Tolls** ($f^{(\omega^*)}$): The flow vector is identical to the new system optimum f^*_{new}: $[6584.51, 5577.70, 3367.56, \ldots, 4905.74]$
- **Wardrop Cost with Tolls (Total Time):** 26186.39
- **PoA with tolls:** 1.0000

This PoA value of 1.0 confirms that the Wardrop equilibrium under these new tolls coincides exactly with the new system optimum. This validates the marginal cost pricing principle, showing that it holds even when the definition of "social cost" is changed.

Conclusion

This homework explored fundamental concepts in network dynamics through three comprehensive exercises. In Exercise 1, we analyzed flow networks and capacity allocation strategies using max-flow min-cut theorems, demonstrating how strategic capacity distribution can maximize network throughput. Exercise 2 investigated centrality measures in directed graphs, revealing how different metrics capture distinct aspects of node importance and influence. Exercise 3 applied these concepts to real-world traffic optimization, showing how toll mechanisms can align individual behavior with social optimality.
The analyses demonstrate the power of network theory in solving complex optimization problems across diverse domains, from abstract graph structures to practical traffic management systems. The Price of Anarchy calculations particularly highlight the trade-off between individual rationality and collective efficiency, and show how carefully designed incentive mechanisms can bridge this gap.